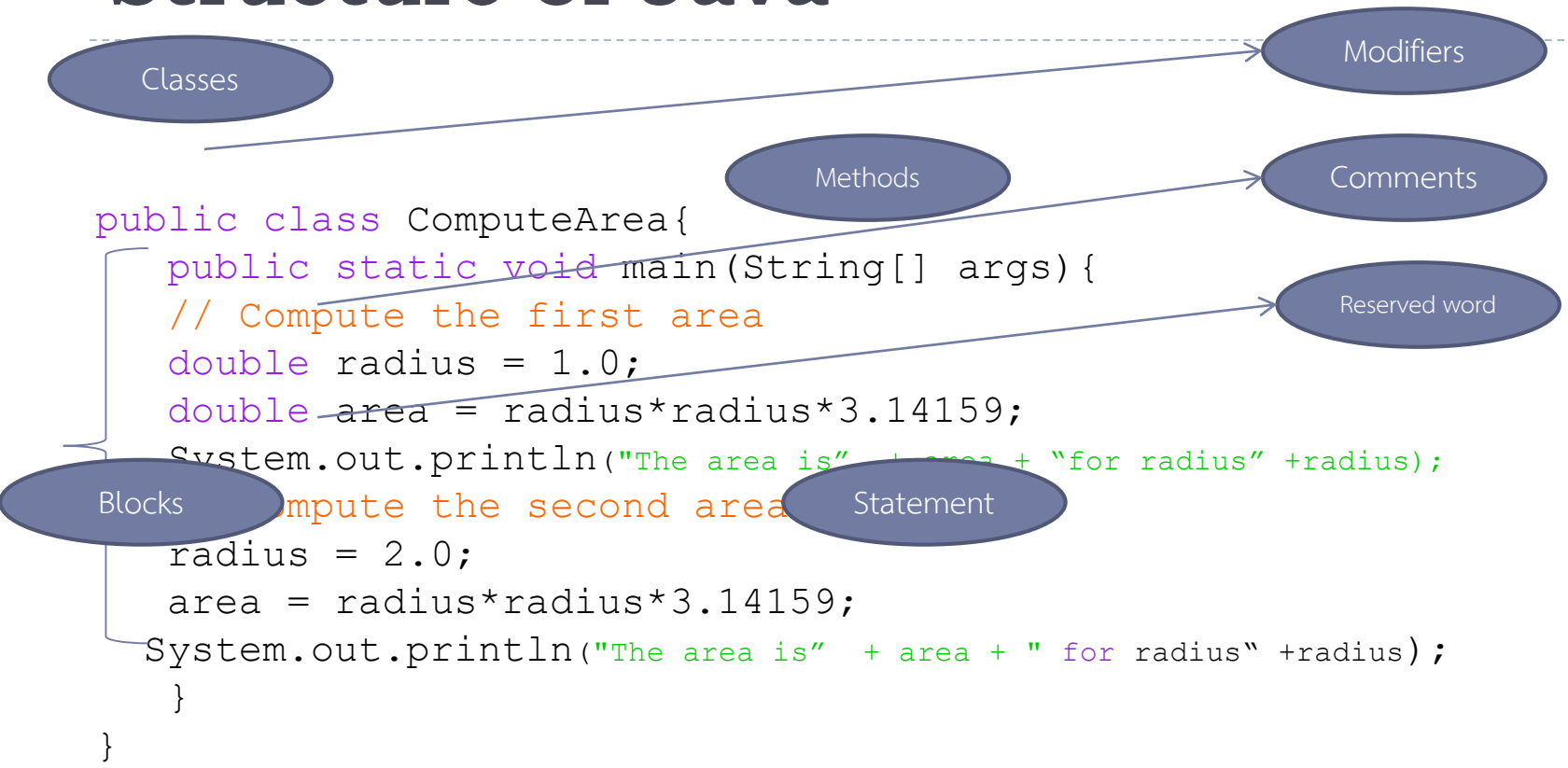
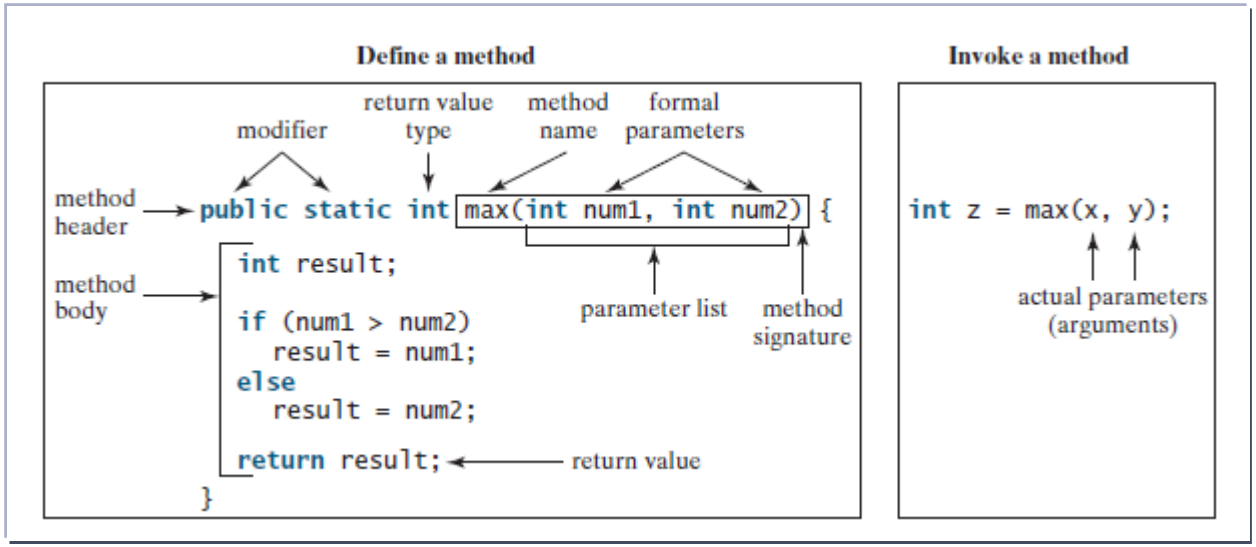
Object Oriented Programming

Lecture#3 Basic of OOP- Method

Structure of Java



Methods



A **method** is a collection of statements that are grouped together to perform an operation.

Method signature is the combination of the *method name* and the *parameter list*.

CAUTION

- A return statement is required for a value-returning method.
- The method shown below is logically correct, but it has a compilation error
  - because the Java compiler thinks it possible that this method does not return any value.

```
public static int xMethod(int n) {
   if (n > 0) return 1;
   else if (n == 0) return 0;
   else return -1;

 }
```

To fix this problem, delete *if (n < 0)* in (a), so that the compiler will see a return statement to be reached regardless of how the if statement is evaluated.

Passing Parameters

```
access_modifier return_type
  methodName(parameter_list) {
  // Method body
  return type;
}
```

A method may return a value. The returnValueType is the data type of the value the method returns. If the method does not return a value, the returnValueType is the keyword void.

```
public static void nPrintln(String message, int n)
  {
   for (int i = 0; i < n; i++)
     System.out.println(message);
   }
```

- The arguments are passed by value to parameters when invoking a method.

Type of declaration of methods

- **Type of declaration of methods based on return type and arguments:**
  - Method without return type and without arguments.
  - Method without return type and with arguments.
  - Method with return type and without arguments.
  - Method with return type and with arguments.

Method with out return type and without arguments.

```
class sample{
  public void add(){
        int a=10;
        int b=10;
        int c=a+b;
        System.out.println(c);
 }
public static void main(String args[]) // ->method prototype.
{
        sample obj= new sample();
        obj.add();
 }

}
```

Method with out return type and with arguments.

```
class sample{
  public void add(int a, int b){
        int c=a+b;
        System.out.println(c);
  }
public static void main(String args[]) // ->method prototype.
{
        sample obj= new sample();
        obj.add(1,2);
 }
}
```

Method with return type and without arguments.

```
class sample{
  public int add(){
        int a=20;
        int b=30;
        int c=a+b;
        return c;
}
public static void main(String args[]) // ->method prototype.
{       sample obj= new sample();
        int x=obj.add();
        System.out.println(x);
 }
}
```

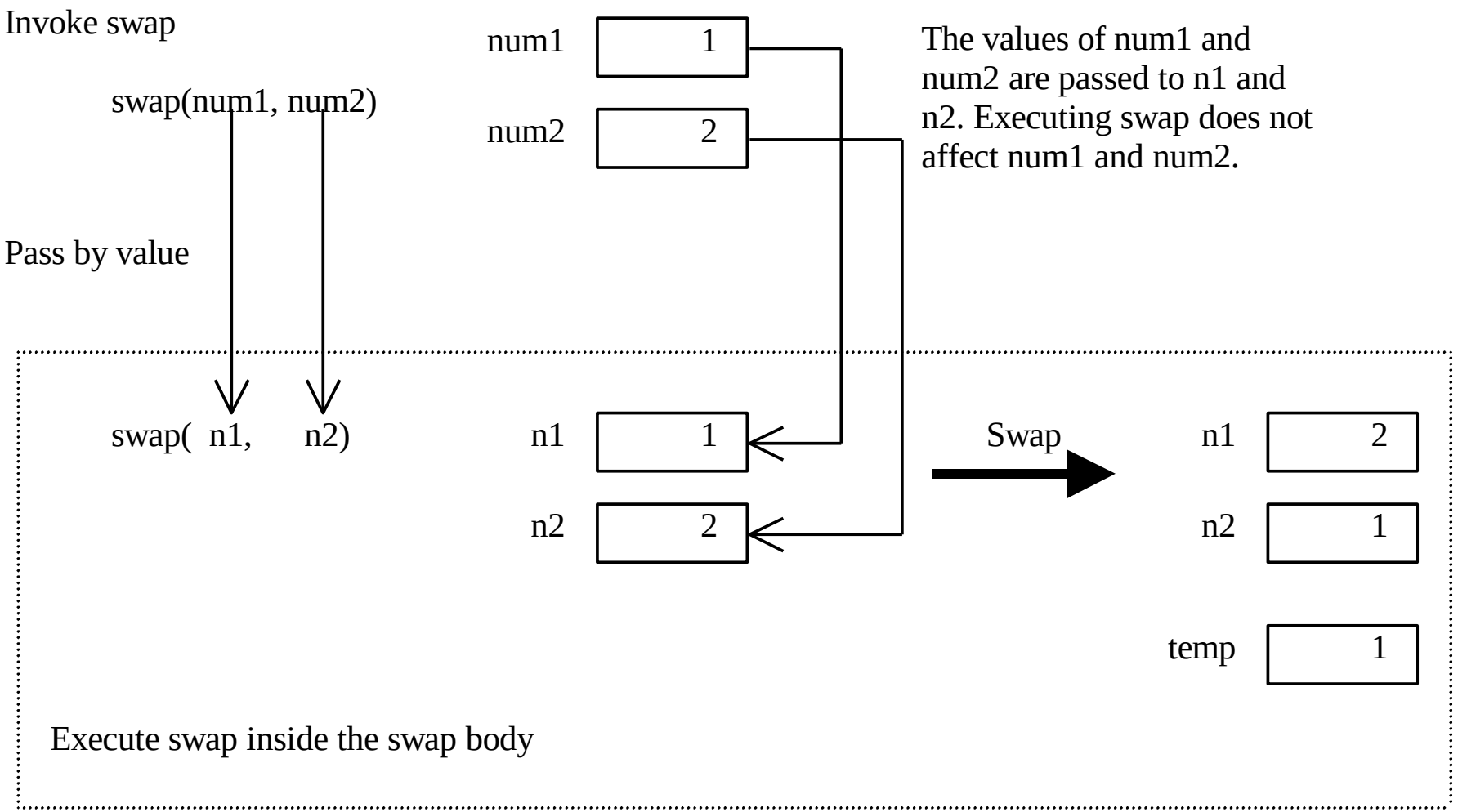
Method with return type and with arguments.

```
class sample{
public int add(int a, int b){
  int c=a+b;
  return c;
}
public static void main(String args[]) // ->method prototype.
{
  sample obj= new sample();
  int x=obj.add(1,2);
  System.out.println(x);
  }
}
```

Pass by Value

```
public class TestPassByValue {
  /** Main method */
  public static void main(String[] args) {
    // Declare and initialize variables
    int num1 = 1;
    int num2 = 2;

    System.out.println("Before invoking the swap method, num1 is " +
      num1 + " and num2 is " + num2);

    // Invoke the swap method to attempt to swap two variables
    swap(num1, num2);

    System.out.println("After invoking the swap method, num1 is " +
      num1 + " and num2 is " + num2);
  }

  /** Swap two variables */
  public static void swap(int n1, int n2) {
    System.out.println("\tInside the swap method");
    System.out.println("\t\tBefore swapping, n1 is " + n1
      + " and n2 is " + n2);

    // Swap n1 with n2
    int temp = n1;
    n1 = n2;
    n2 = temp;

    System.out.println("\t\tAfter swapping, n1 is " + n1
      + " and n2 is " + n2);
  }
}
```

Pass by Value



Passing Objects to Methods

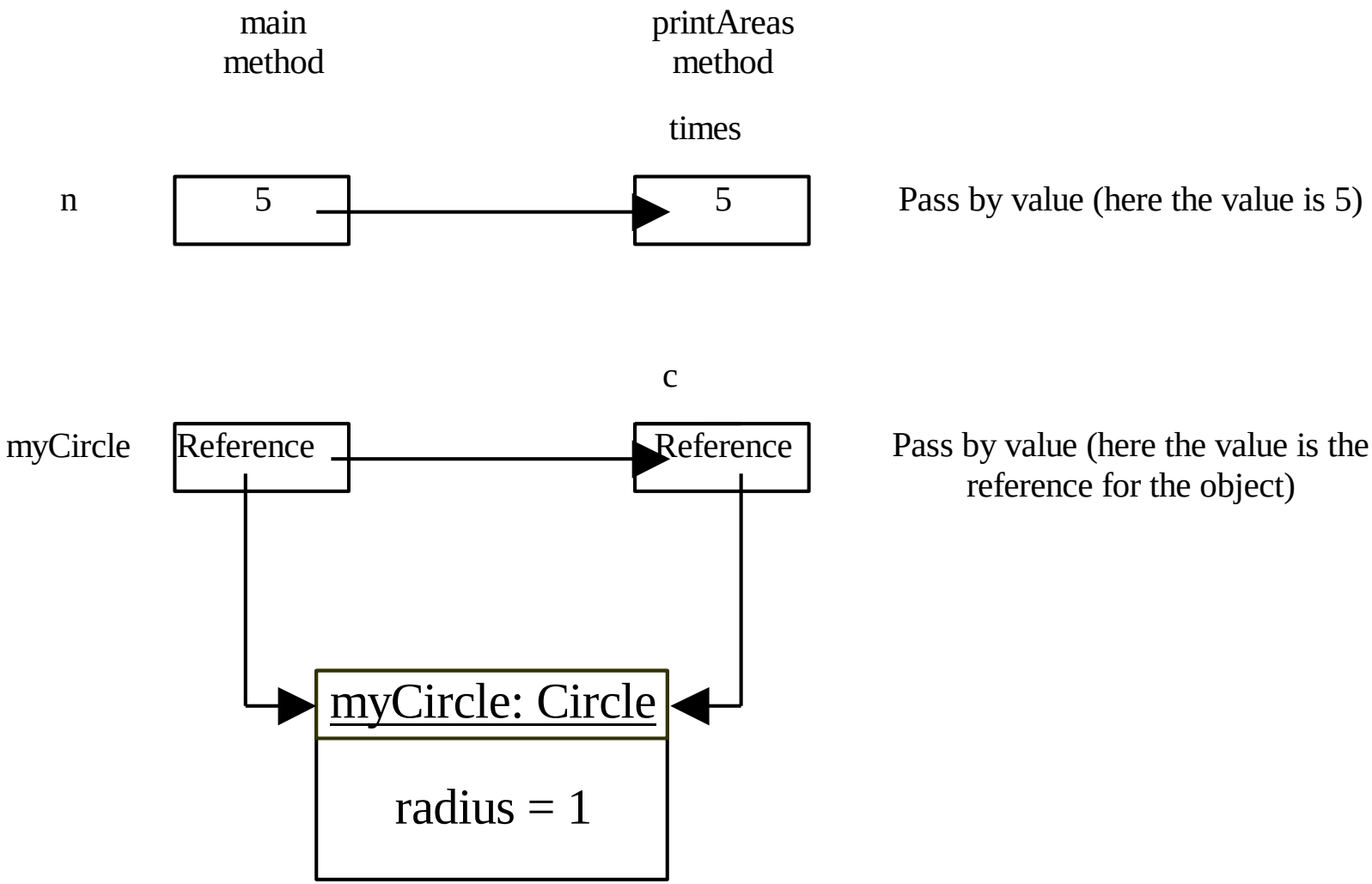
- Example Passing Objects as Arguments

```
public class TestPassObject {
  /** Main method */
  public static void main(String[] args) {
    // Create a Circle object with radius 1
    Circle3 myCircle = new Circle3(1);

    // Print areas for radius 1, 2, 3, 4, and 5.
    int n = 5;
    printAreas(myCircle, n);

    // See myCircle.radius and times
    System.out.println("\n" + "Radius is " + myCircle.getRadius());
    System.out.println("n is " + n);
  }
  /** Print a table of areas for radius */
  public static void printAreas(Circle3 c, int times) {
    System.out.println("Radius \t\tArea");
    while (times >= 1) {
      System.out.println(c.getRadius() + "\t\t" + c.getArea());
      c.setRadius(c.getRadius() + 1);
      times--;
    }
  }
}
```

Passing Objects to Methods

```
public class Circle3 {
  /** The radius of the circle */
  private double radius = 1;

  /** The number of the objects created */
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public Circle3() {
    numberOfObjects++;
  }

  /** Construct a circle with a specified radius */
  public Circle3(double newRadius) {
    radius = newRadius;
    numberOfObjects++;
  }

  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius) {
    radius = (newRadius >= 0) ? newRadius : 0;
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double getArea() {
  return radius * radius * Math.PI;
  }
}
```

Passing Objects to Methods



Method Overloading

- Overloading methods enables you to define the methods with the same name as long.

```
public class TestMethodOverloading {
  /** Main method */
  public static void main(String[] args) {
    // Invoke the max method with int parameters
    System.out.println("The maximum of 3 and 4 is "
      + max(3, 4));

    // Invoke the max method with the double parameters
    System.out.println("The maximum of 3.0 and 5.4 is "
      + max(3.0, 5.4));

    // Invoke the max method with three double parameters
    System.out.println("The maximum of 3.0, 5.4, and 10.14 is "
      + max(3.0, 5.4, 10.14));
  }

  /** Return the max of two int values */
  public static int max(int num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }

  /** Find the max of two double values */
  public static double max(double num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }

  /** Return the max of three double values */
  public static double max(double num1, double num2, double num3) {
    return max(max(num1, num2), num3);
  }
}
```

Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match.
- This is referred to as *ambiguous invocation*.
- Ambiguous invocation is a compilation error.

Ambiguous Invocation

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }
 public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
 public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Scope of Local Variables

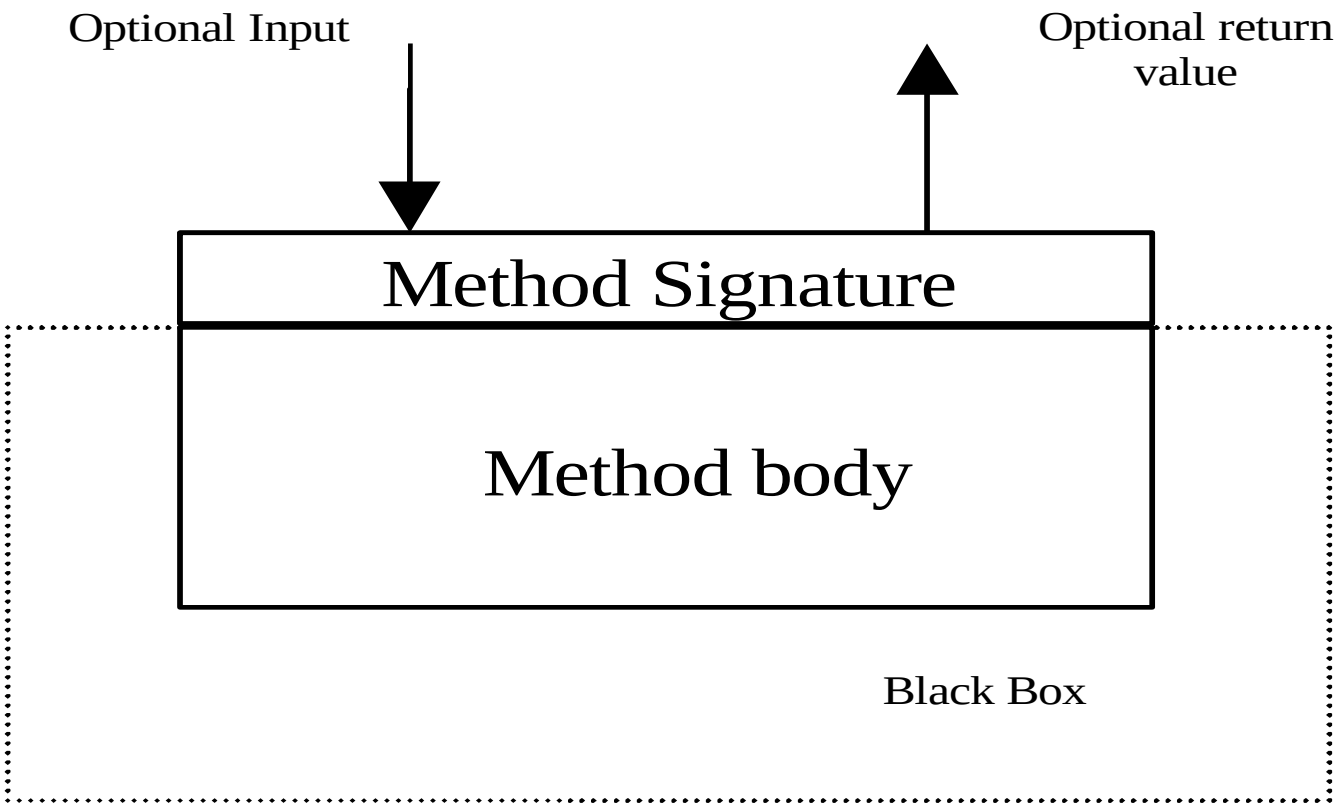
```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
// With no errors
public static void incorrectMethod() {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    int x = 0;
    x += i;
  }
}
```

Scope of Local Variables

- A local variable:
  - a variable defined inside a method.
- Scope:
  - the part of the program where the variable can be referenced.
- The scope of a local variable
  - starts from its declaration and continues to the end of the block that contains the variable.
- A local variable must be declared before it can be used.

Method Abstraction

You can think of the method body as a black box that contains the detailed implementation for the method.



Benefits of Methods

- Write a method once and reuse it anywhere.
- Information hiding. Hide the implementation from the user.
- Reduce complexity.

The **Math Class**

The **Math class contains the methods needed to perform basic mathematical functions**

```
public class Math_method  {
public static void main (String args[]) {
   System.out.println("Natural logarithm of 10 : "+Math.log(10) );
   System.out.println("Exponential of 1.0 : " + Math.exp(1.0) );
   System.out.println("The third power of 2 : " + Math.pow(2, 3) );
   System.out.println("Square root of 7 : " + Math.sqrt(7) );
   System.out.println("Absolute value of -10 : "+Math.abs(-10) );
   System.out.println("Ceiling value of 9.2 : " + Math.ceil(9.2) );
   System.out.println("Floor value of 9.2 : " + Math.floor(9.2) );
   System.out.println("Maximum of 2 and 3 : " + Math.max(2,3) );
   System.out.println("Minimum of 2 and 3 : " + Math.min(2,3) );
   System.out.println("Sine of 0 radian : " + Math.sin(0.0) );
   System.out.println("Cosine of 0 radian : " + Math.cos(0.0) );
   System.out.println("Tangent of 0 radian : " + Math.tan(0.0) );
   System.out.println("Pi : " + Math.PI );
   System.out.println("Random number (0.0 to 1.0) : " + Math.random() );
}
}
```

The `Math` Class

- Class constants:
  - `PI`
  - `E`
- Class methods:
  - Trigonometric Methods
    - `sin(double a)`
    - `cos(double a)`
    - `tan(double a)`
    - `acos(double a)`
    - `asin(double a)`
    - `atan(double a)`
  - min, max, abs, and random Methods
    - `max(a, b)` and `min(a, b)`
    - `abs(a)`
    - `random()`
  - Exponent Methods
    - `exp(double a)` `e` raised to the power of `a`.
    - `log(double a)` natural logarithm of `a`.
    - `pow(double a, double b)` `a` raised to the power of `b`.
    - `sqrt(double a)`
  - Rounding Methods
    - `double ceil(double x)` `x` rounded up to its nearest integer.
    - `double floor(double x)` `x` is rounded down to its nearest integer.
    - `double rint(double x)` `x` is rounded to its nearest integer. If `x` is equally close to two integers, the even one is returned as a double.
    - `int round(float x)` Return `(int)Math.floor(x+0.5)`.
    - `long round(double x)` Return `(long)Math.floor(x+0.5)`.